AiWingman: Design, Implementation, and an Open Evaluation Protocol for a Local Work-Continuity Companion for Parallel Coding-Agent Tasks

Mehmet Solak

Department of Biosystems Engineering, Faculty of Agriculture, Siirt University, Siirt, Türkiye

ORCID: 0000-0003-2528-7960 | Correspondence: mehmetsolak@siirt.edu.tr

Draft 0.1 - 23 August 2026 - Author confirmation required before public deposit

Abstract

Parallel coding-agent tasks can produce a fragmented work surface: requests for input, completed results, blocked work, and quiet but unfinished tasks may coexist across many windows. Timestamps alone do not establish whether a task is running, finished, abandoned, or worth resuming. This technical report presents AiWingman, an open-source macOS menu-bar companion that reads locally retained Codex task state through read-only adapters, derives bounded and explainable continuity signals, and returns the user to a selected task through a local deep link. The design separates observations from user decisions, treats silence as neutral, and abstains from ranking when evidence is incomplete or insufficient. A distinct optional Wingman flow can send a previewed and bounded packet to a separately installed Codex CLI only after one-shot consent; this remote path is not part of the offline dashboard and is not presented as an isolation guarantee. The public source release was subjected to regression and contract checks on arm64 and native x86_64 macOS runners. These checks establish only the enumerated engineering properties; they do not establish human benefit, scientific superiority, exact token waste, full privacy isolation, or cross-system compatibility. The report therefore contributes a working software artifact, an auditable claim boundary, and an open protocol for later conformance, robustness, privacy, determinism, and performance evaluation. Human efficacy remains untested.

Keywords: coding agents; work continuity; local software; privacy engineering; explainable triage; abstention; software artifact

1. Introduction

Coding agents increasingly operate through persistent tasks rather than a single linear dialogue. One person may start several tasks, delegate subtasks, wait for tool results, answer an agent's question, and later return to an older line of work. This interaction pattern raises a continuity concern. The next useful task is not necessarily the newest task, and a quiet task is not necessarily abandoned. Cognitive studies of interrupted work motivate the importance of goal activation and resumption cues [2,3], but they do not by themselves show that a particular dashboard improves human performance.

AiWingman was built as a local companion for this narrow problem. It is not a coding agent, an autonomous task manager, or a replacement for Codex. Its ordinary path reads locally retained task metadata and bounded rollout evidence, computes descriptive signals, shows why a task may deserve attention, and opens the chosen task back in Codex. It also provides explicit and reversible lifecycle controls for states such as obsolete or abandoned. The user, not elapsed time, makes those decisions.

The project follows a source-first distribution model. The evaluated artifact is the public `v1.2.0-beta.2` source release at commit `5e212181ae177cd555ab6bb92f5f71ac8be9173a`. The software is MIT licensed. This report is prepared for open distribution under CC BY 4.0 after author approval. The software is an independent community project and is not an official OpenAI product.

This report makes three bounded contributions:

- A native implementation that joins read-only local task evidence, explicit continuity state, explainable triage, and return-to-task navigation.
- A privacy and inference contract that separates the offline dashboard from the optional consent-gated remote review and records important residual risks.
- A public evaluation protocol that states what future conformance, robustness, privacy, determinism, and performance evidence must contain before stronger claims can be made.

The report is intentionally not an efficacy study. It uses no recruited participants, surveys, interviews, private task histories, or outcome measurements. It does not claim faster resumption, lower cognitive load, fewer wasted tokens, or superiority over another tool.

2. Related Work and Positioning

Local-first software argues that users should retain control of their data and remain able to work without a service dependency [1]. AiWingman's ordinary dashboard follows a narrower local-data principle: task evidence is read from files already present on the user's Mac, analysis runs locally, and the dashboard has no service component. The optional Wingman critique is deliberately excluded from that claim because it starts a separate remote Codex CLI turn after consent.

Research on interrupted tasks describes how goals decay and how preparation or cues can assist task resumption [2,3]. AiWingman uses this literature as motivation for exposing checkpoints, next actions, waiting conditions, and evidence-backed attention reasons. No direct transfer from laboratory findings to coding-agent work is assumed, and no human-outcome claim is made.

Code-oriented language models and agent benchmarks have emphasized generated code, issue resolution, and agent-computer interfaces [4-6]. AiWingman addresses a different layer: continuity across the human operator's set of agent tasks. It does not score code correctness or resolve repository issues autonomously. Its signals are descriptive views of local task evidence, not a benchmark of agent capability.

The design also relates conceptually to systems that may abstain when their evidence does not support a reliable selection [8]. AiWingman's abstention is a deterministic product policy rather than a learned selective-prediction model: the interface returns no ranked continuity candidates when history is partial, evidence is weak, top candidates are too close, or the user has deferred the work.

Finally, the artifact is versioned and cited as software, consistent with the principle that research software should be identifiable and citable [7]. Publication of a source release is nevertheless not equivalent to peer review, scientific validation, or confirmation of novelty.

3. Design Requirements

The implementation is organized around six requirements.

3.1 Preserve source state

Codex state under `~/.codex` is treated as external and read-only. SQLite adapters use read-only open flags and query-only mode. AiWingman does not repair, migrate, or write Codex databases. An unsupported schema produces a neutral compatibility error.

3.2 Separate observation from interpretation

Observable events such as a request for input, a final answer, recent activity, a blocker, or partial history are distinct from lifecycle decisions. Inactivity is not execution evidence and is not mapped automatically to obsolete or abandoned. High-impact lifecycle labels require an explicit and reversible user action.

3.3 Make ranking explainable and optional

Continuity candidates carry visible reasons such as unseen result, explicit input request, or a saved next action. At most three candidates are shown. Ranking abstains when available evidence is incomplete or too ambiguous to support a useful ordering.

3.4 Bound content and computation

Rollout sampling and remote review packets have explicit size and count limits. Incomplete local evidence remains marked as partial. The system does not silently convert missing evidence into a negative finding.

3.5 Keep remote review distinct

The dashboard starts no background agent call. A Wingman critique is a separate user-triggered action with an exact packet preview, a fixed review instruction and output schema, one-shot consent, and an ephemeral CLI turn. Consent is cleared after each attempt and when scope settings change.

3.6 Preserve legacy user data

The public product name is AiWingman, while selected executable, bundle, preference, and application-support identifiers retain the earlier Activity Radar names. This is a deliberate compatibility measure so an upgrade does not discard existing local state.

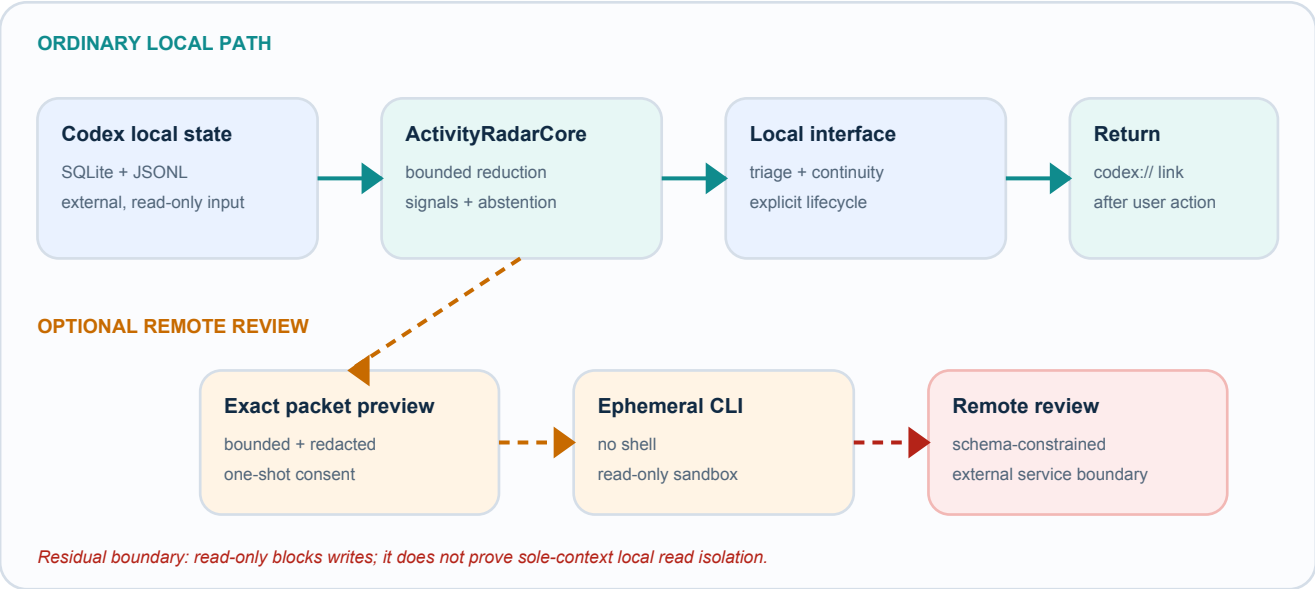

Figure 1. AiWingman architecture and trust boundaries. Solid arrows are ordinary local paths. The dashed branch is optional, remote, and consent-gated.

4. Architecture and Data Flow

AiWingman is a native Swift application targeting macOS 13 or later. Its interface uses SwiftUI and AppKit and runs as an `LSUIElement` menu-bar application. The package exposes four products: `ActivityRadarCore`, the `ActivityRadar` application, `ActivityRadarDiagnostics`, and `ActivityRadarSelfTest`.

`ActivityRadarCore` contains read-only Codex adapters, rollout reduction, task and theme analysis, attention and lifecycle policy, continuity triage, and bounded Wingman evidence extraction. The application target contains the menu-bar and command-palette interface, local preferences, continuity storage, deep-link navigation, and the optional CLI process boundary. Diagnostics emits aggregate compatibility information without task content. The self-test executable makes deterministic contract checks available when full test discovery is not available.

The ordinary path has four stages. First, the adapter opens the local Codex SQLite database in read-only mode and resolves a defensive parent-child forest. Spawned subagent tasks are grouped under the user-owned root task. Second, bounded rollout sampling reduces relevant events to a local state representation while retaining a partial-history flag. Third, deterministic policies compute attention reasons, explicit lifecycle state, continuity cues, and limited descriptive graph signals. Fourth, the interface shows the result and may hand a `codex://threads/<thread-id>` URL to macOS after the user chooses a task.

AiWingman-owned state is separate from Codex state. Interface language, date range, navigation state, and lifecycle choices are stored in local preferences. Continuity records are stored under the legacy application-support path. The store rejects any root beneath `~/.codex`, uses restrictive directory and file modes, pseudonymizes task identifiers, and bounds the optional content-free research ledger by age and count.

5. Evidence Model and Conservative Triage

5.1 Task trees and token counter proxy

Spawned rollout files can share a cumulative token-counter lineage. Adding the root and descendant counters would therefore double count shared history. AiWingman represents each task tree by the largest cumulative counter observed in that tree and uses it as a comparison proxy. This value is not an exact billed-token total, a period-specific total, or a measure of waste. The date filter selects trees by latest activity; it does not transform the lifetime proxy into consumption within the selected period.

The interface can surface candidates for review, such as large trees with unresolved or repeatedly restarted activity. These are prompts for human inspection. It cannot infer that tokens were wasted, that a prompt was poor, or that a task lacked value.

5.2 Descriptive graph signals

Task and theme relationships use deterministic text features and graph summaries such as similarity, weighted degree, PageRank, and connected components. These quantities describe the retained corpus. They are not measures of task success, scientific interest, personality, cognitive load, or causal importance. Prompt-derived text signals are kept local unless the user separately opts to include them in a previewed Wingman packet.

5.3 Attention and lifecycle

The state model distinguishes observations from user-owned decisions. Examples of observations include explicit input requested, blocker observed, unseen result, recent activity, quiet open work, and partial history. Examples of decisions include snoozed, waiting on an external event, explicitly obsolete, and explicitly abandoned. Obsolete and abandoned remain reversible labels. Silence alone has no lifecycle meaning.

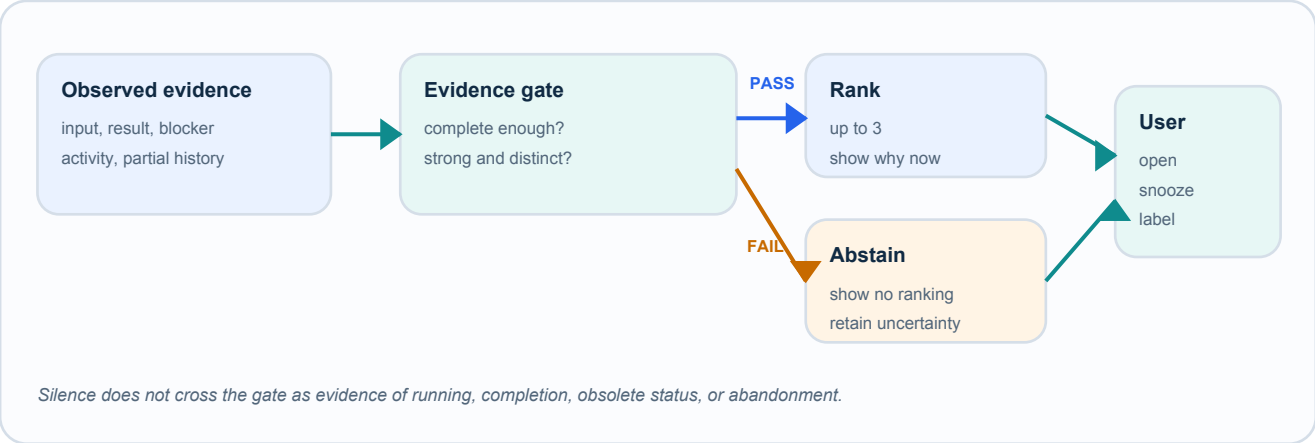

Figure 2. Conservative triage. Missing or weak evidence leads to abstention rather than a fabricated priority.

6. Optional Wingman Review and Threat Boundary

The optional Wingman flow exists to critique prompting and harness scope, summarize selected work, and identify task trees that may deserve a manual token review. It is not attached to the currently open Codex task and it is not required for the dashboard.

Before every invocation, AiWingman shows the exact user-derived JSON packet intended for standard input. Local analysis can select up to 20 task trees; the remote packet includes detailed rows for at most 12 and states how many were selected, detailed, and omitted. Raw task identifiers, full paths, working and rollout paths, git and account metadata, system and developer instructions, tool outputs, and credentials are excluded. Prompt excerpts, prompt-derived themes, and local text signals are off by default and share one explicit opt-in control. Human-authored continuity text is never included.

After one-shot consent, the application invokes a separately installed and signed-in Codex CLI directly, without a shell. The child process uses approval-disabled, read-only sandboxing, bounded input, output, and time, an ephemeral turn, ignored user configuration, and a strict output schema. Unknown or tool events fail closed. A validated authentication file is copied opaquely into a private temporary CLI home and removed after the attempt; its contents are not parsed or added to the packet.

These controls do not prove sole-context isolation. Read-only sandboxing prevents writes but does not prove that the child process cannot read another local file. `--ephemeral` prevents a local rollout from being saved but does not specify service-side retention. The preview is therefore an exact statement of intended user-derived packet content, not a guarantee that the packet is the only technically accessible context. Remote output is also excluded from local determinism claims.

7. Current Engineering Verification

The public release was checked by GitHub Actions run `32648392604` on 23 August 2026. Two jobs completed successfully: `macOS verification` on an arm64 macOS 15 runner and `Intel runtime` on a native x86_64 macOS 15 runner. Each architecture ran 52 Swift tests and 16 deterministic self-tests. The native Intel job also built the release application. The arm64 public-source gate additionally checked the read-only/privacy contract, content-free diagnostics, continuity-store privacy and retention behavior, manifest consistency, a release build, and a locally packaged Universal 2 application.

Table 1. Current automated engineering evidence. A passing job is not an independent benchmark, a notarized public binary, or a clean-machine acceptance result.

Evidence class	arm64 macOS 15	native x86_64 macOS 15	Supported interpretation
Swift test suite	52 passed	52 passed	Regression checks passed at the evaluated revision
Core self-tests	16 passed	16 passed	Enumerated deterministic contracts passed
Release build	Passed	Passed	Source compiled for both tested architectures
Content-free diagnostics fixture	Passed	Not separately run in that job	Fixture output excluded seeded private sentinels
Continuity-store privacy and retention harness	Passed	Not separately run in that job	Enumerated local storage checks passed
Universal 2 local package verification	Passed	Architecture built natively	Both slices were present in the locally built bundle

The current source release contains no prebuilt application asset. It has not supplied publication-grade independent conformance, adversarial robustness, privacy-isolation, determinism, or performance results. The declared macOS 13 deployment target is not evidence that the evaluated revision was run on macOS 13. No signed and notarized binary is claimed in this report.

Table 2. Claim ledger. Unsupported claims are part of the artifact's explicit boundary rather than omitted negative evidence.

Claim	Status at report version 0.1	Boundary
Working open-source implementation	Supported	Exact public tag and commit are identified
Read-only source adapter contract	Regression-tested	Not an external security proof
Explainable local triage with abstention	Implemented and regression-tested	No human-benefit claim
Optional previewed, consent-gated review	Implemented and contract-tested	No sole-context or service-retention guarantee
Exact token waste measurement	Not supported	Only a cumulative tree-level comparison proxy is available
Automatic obsolete or abandoned inference	Intentionally not supported	These are explicit, reversible user choices
Human efficacy	Untested	No participants or outcome measures
Comparative superiority	Not supported	No baseline benchmark has been run
Publication-grade conformance and robustness	Awaiting frozen protocol	Current tests are engineering regression evidence

8. Open Technical Evaluation Protocol

The repository includes a publication blueprint, but its independent specification, synthetic corpus, and oracle are not yet frozen. Accordingly, this section describes a protocol rather than completed results. A future evaluated release should freeze the following materials before running the benchmark: a normative state and output specification; a licensed synthetic and adversarial corpus; independent expected outputs; seeds and generators; an environment matrix; a machine-readable result schema; and explicit stop rules.

8.1 Conformance

Normative fixtures should cover turn lifecycle, pending input resolution, final answers, blockers, explicit lifecycle actions, parent-child grouping, partial-history propagation, date-range behavior, neutral incompatibility, and return-to-task link formation. Expected outputs must be derived from the frozen specification rather than copied from the implementation.

8.2 Robustness and compatibility

Adversarial fixtures should include truncated and malformed JSONL, missing files, stale indices, duplicate or cyclic spawn edges, unsupported schemas, extreme integer values, invalid UTF-8 boundaries, and interrupted reads. The denominator and every failure must be retained. A crash, unbounded hang, source mutation, or silent acceptance of an unsafe schema should fail the relevant claim.

8.3 Privacy boundary

Fixtures should seed unique sentinels into raw identifiers, titles, messages, paths, prompts, checkpoints, tool outputs, credentials, and configuration. Automated gates should then inspect diagnostics, research exports, logs, temporary homes, preview packets, and remote request construction for forbidden sentinels. These tests can support only the enumerated non-disclosure properties; they cannot prove that an external process has no other readable context.

8.4 Determinism

Local analysis should be repeated on identical frozen inputs across multiple processes and supported architectures. Discrete outputs, abstention decisions, ordering, and serialized artifacts should be compared. Remote model output must be reported separately and excluded from local determinism totals.

8.5 Performance

Synthetic corpora should vary root-task count, spawn depth, rollout size, partial-history rate, and text volume. Measurements should report median and p95 latency, peak memory, repetitions, toolchain, operating system, and hardware. Performance measurements describe computation on synthetic workloads and cannot be converted into human time saved.

Table 3. Future benchmark matrix. No row is presented as completed publication evidence in this report.

Axis	Frozen input	Required result	Kill condition
Conformance	Versioned normative fixtures and oracle	Full pass/fail ledger	Any must-pass fixture fails
Robustness	Malformed, partial, extreme, and incompatible fixtures	Complete outcomes and failure logs	Crash, hang, mutation, or unsafe silent acceptance
Privacy	Synthetic sentinels in every forbidden field	Artifact-by-artifact disclosure matrix	Any forbidden sentinel escapes
Determinism	Identical frozen inputs and repetitions	Exact discrete-output comparison	Unexplained local divergence
Performance	Parameterized synthetic workloads	Median, p95, memory, environment	Missing denominator or environment

9. Limitations and Threats to Validity

AiWingman depends on local Codex files and deep-link behavior that are not a stable public integration contract. A future Codex release may change filenames, schemas, event shapes, or URL routes. Neutral failure reduces the risk of source mutation but does not remove compatibility risk.

The evidence model is intentionally incomplete. Bounded rollout sampling can miss context; partial history is exposed but cannot reconstruct absent events. Task grouping depends on locally available spawn relationships. Text similarity and graph summaries can be sensitive to preprocessing choices and should not be interpreted causally.

The token counter is a comparison proxy based on the largest observed cumulative counter in a task tree. It may differ from account billing, model-side accounting, and the user's concept of useful or wasted work. No cost or waste estimate is validated.

The remote Wingman path depends on an external CLI and service. Packet minimization, preview, one-shot consent, and process controls narrow intended disclosure, but local read access and service-side processing remain outside the proof boundary. Users must inspect the packet and current service terms before consenting.

Current testing is implementation-authored regression evidence. The publication specification, corpus, and oracle remain unfrozen, and the artifact has not undergone an independent security audit. CI coverage on arm64 and native x86_64 macOS 15 does not establish operation on all supported macOS versions or hardware.

No participant data were collected. This avoids claims unsupported by a short field phase, but it also means the report cannot answer whether AiWingman improves recall, resumption time, decision quality, workload, or token use in practice.

10. Reproducibility and Availability

Source code, documentation, tests, security policy, privacy boundary, contribution guide, and citation metadata are publicly available at:

`https://github.com/mehmetsolakedu/activity-radar`

The exact artifact described here is release `v1.2.0-beta.2`, commit `5e212181ae177cd555ab6bb92f5f71ac8be9173a`:

`https://github.com/mehmetsolakedu/activity-radar/releases/tag/v1.2.0-beta.2`

The evaluated CI record is:

```
https://github.com/mehmetsolakedu/activity-radar/actions/runs/32648392604
```

The repository is licensed under MIT. Public examples, fixtures, screenshots, and future benchmark artifacts must be synthetic and rights-cleared. Private Codex databases, rollout files, task text, local paths, credentials, and real-work screenshots are outside the publication package.

11. Declarations

Ethics and data statement. This report describes software architecture and engineering checks. It collected no human-participant, animal, survey, interview, or private task-history data. No inference about user efficacy is made.

Funding statement. Author confirmation is required before public deposit.

Competing interests. Author confirmation is required before public deposit.

Author contribution. Mehmet Solak conceived the product direction, defined the intended use, supervised iterative development, and is the sole named author. Final wording requires author confirmation before public deposit.

AI assistance statement. OpenAI Codex assisted with source discovery, manuscript structuring, initial language drafting, code inspection, and document formatting. Before deposit, the named author must verify every claim and reference, revise the prose as needed, and assume full responsibility. AI is not listed as an author.

License statement. The software is distributed under the MIT License. This manuscript is prepared for distribution under the Creative Commons Attribution 4.0 International license after author approval.

References

[1] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan, "Local-first software: You own your data, in spite of the cloud," Proceedings of the ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software, pp. 154-178, 2019. https://doi.org/10.1145/3359591.3359737

[2] E. M. Altmann and J. G. Trafton, "Memory for goals: An activation-based model," Cognitive Science, vol. 26, no. 1, pp. 39-83, 2002. https://doi.org/10.1207/S15516709COG2601_2

[3] J. G. Trafton, E. M. Altmann, D. P. Brock, and F. E. Mintz, "Preparing to resume an interrupted task: Effects of prospective goal encoding and retrospective rehearsal," International Journal of Human-Computer Studies, vol. 58, no. 5, pp. 583-603, 2003. https://doi.org/10.1016/S1071-5819(03)00023-5

[4] M. Chen et al., "Evaluating large language models trained on code," arXiv:2107.03374, 2021. https://doi.org/10.48550/arXiv.2107.03374

[5] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" International Conference on Learning Representations, 2024. https://doi.org/10.48550/arXiv.2310.06770

[6] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," Advances in Neural Information Processing Systems, vol. 37, pp. 50528-50652, 2024. https://doi.org/10.52202/079017-1601

[7] A. M. Smith, D. S. Katz, K. E. Niemeyer, and FORCE11 Software Citation Working Group, "Software citation principles," PeerJ Computer Science, vol. 2, e86, 2016. https://doi.org/10.7717/peerj-cs.86

[8] Y. Geifman and R. El-Yaniv, "SelectiveNet: A deep neural network with an integrated reject option," Proceedings of the 36th International Conference on Machine Learning, PMLR 97, pp. 2151-2159, 2019. https://doi.org/10.48550/arXiv.1901.09192